

HelixQA

HelixQA

Orquestación anti-engaño de QA — sesiones autónomas multiplataforma donde cada APROBADO incluye evidencia capturada de que un usuario real puede utilizar la función.

Resumen

HelixQA es un marco de orquestación anti-engaño para pruebas multiplataforma (Android, Android TV, Web, Escritorio) que combina bancos de pruebas YAML, detección de fallos en tiempo real, captura de evidencia paso a paso y sesiones autónomas de QA con LLM y visión por computadora para demostrar que las funciones funcionan de extremo a extremo. Es el tipo de prueba de QA obligatorio según Constitution (§11.4.169).

Descripción breve

Un orquestador anti-engaño de QA (Go) que ejecuta bancos de pruebas escritos y sesiones de QA completamente autónomas, impulsadas por LLM y visión por computadora, en múltiples plataformas. Detecta fallos, valida cada paso con evidencia capturada (capturas de pantalla, logcat, video, trazas de pila) y genera automáticamente tickets ricos en evidencia para las canalizaciones de corrección AI.

Descripción detallada

HelixQA es un marco Go cuyo único y inquebrantable principio de diseño se rige por la Regla Operativa §11.4 de Constitution: el estándar para lanzar un producto no es “las pruebas pasan”, sino “los usuarios pueden usar la función”. Por ello, cada APROBADO que emite debe incluir evidencia positiva capturada durante la

ejecución —sin evidencia, no hay luz verde, sin excepciones—. Opera en dos modos complementarios que cubren tanto lo scriptado como lo desconocido.

En primer lugar, **bancos de pruebas escritos**: suites YAML de casos TC-XXX con segmentación por plataforma, prioridad, pasos ordenados (nombre/acción/resultado esperado), etiquetas y referencias documentales. Estos se ejecutan con validación por paso, detección de fallos/ANR en tiempo real (ADB para Android, monitoreo de procesos para web/escritorio), recolección centralizada de evidencia y generación automática de tickets en Markdown, ya adaptados para las canalizaciones de corrección AI.

En segundo lugar, una **sesión de QA completamente autónoma** que entrega la aplicación a agentes impulsados por LLM y visión por computadora, permitiéndoles operarla sin supervisión a través de cuatro fases disciplinadas: configuración (selección de LLMs, creación de un mapa de funciones a partir de la documentación del proyecto, despliegue de agentes CLI, inicialización del motor de visión), verificación guiada por documentación que recorre cada función documentada, exploración basada en curiosidad que prueba deliberadamente casos límite y comportamientos no documentados, y finalmente, generación de informes y limpieza en Markdown/HTML/JSON, vinculando cada hallazgo a evidencia con marca de tiempo en video.

Lo crucial es que no califica su propio trabajo: integra cuatro submódulos externos Go (LLMsVerifier, LLMOrchestrator, VisionEngine, DocProcessor) y reutiliza la infraestructura compartida de challenges y containers, de modo que el componente que navega por la aplicación no es el mismo que juzga si funcionó. Su propia suite se somete al mismo estándar que impone a otros mediante make anti-bluff (escaneo estático + manifiesto de anclaje de comportamiento + mecanismo de mutación) y un Challenge de orquestador de 8 fases con una mutación integrada en §1.1. Una matriz de cobertura de tipos de prueba de 15 filas vincula cada capacidad anunciada a un activo ejecutable concreto y a un formato específico de evidencia capturada, garantizando que las afirmaciones del marco sobre sí mismo están tan respaldadas por evidencia como los veredictos que emite sobre los productos que prueba.

Por qué lo creamos

La QA convencional da luz verde al “aserto superado”, que es exactamente cómo se cuela la clase de fallo que el Constitution denomina *farol*: una función reportada como funcional cuando en realidad está rota para el usuario real. El HelixQA se diseñó para imposibilitar esto en la QA: se niega a calificar como APROBADO sin pruebas físicas (captura de pantalla, logcat, vídeo, traza de pila, informe) obtenidas durante la ejecución real, y trata un

resumen en verde sin dichas pruebas como un defecto crítico equivalente a una funcionalidad ausente. Además, resuelve el problema de la mano de obra —la QA manual exhaustiva en múltiples plataformas no escala— al hacer que las sesiones sean completamente autónomas.

Por qué es un cambio de juego

Fusiona dos elementos que casi nunca conviven en una misma herramienta: un control de calidad riguroso respaldado por pruebas y una exploración autónoma y autoguiada. Un agente LLM con visión abre la *aplicación real*, verifica cada funcionalidad documentada, rastrea los errores no documentados para los que nadie escribió una prueba y genera un rastro de pruebas de calidad judicial mientras lo hace —de modo que “lo probamos” se sustituye por “aquí está el vídeo, aquí el logcat, aquí el ticket”. Y, al ser el submódulo de QA denominado por el Constitution, adoptarlo no mejora la honestidad de la QA para un solo equipo, sino que eleva el estándar para todos los productos de la familia en un solo movimiento.

Qué tiene de innovador

- **Contrato de pruebas antifarol:** cada verificación APROBADA está vinculada a pruebas de ejecución capturadas; una línea verde en CI se considera necesaria, pero nunca suficiente, y un resumen en verde sin pruebas se califica como un defecto crítico.
- **Exploración autónoma guiada por documentación + curiosidad:** verifica cada funcionalidad documentada y luego se sale del guion, probando los casos límite que encuentran los usuarios reales (entradas vacías, interacciones rápidas, rutas no documentadas) que ninguna suite escrita a mano anticipó.
- **Oráculo visual:** la visión mecánica de GoCV junto con el LLM Vision API literalmente ve la interfaz en pantalla, detectando estados visualmente rotos que pasan desapercibidos para asertos basados en tokens o propiedades.
- **Bancos de pruebas estructurados, no en prosa:** las cadenas del banco describen estructura y generan preguntas dinámicas en tiempo de ejecución (CONST-046), de modo que un solo banco funciona en todos los idiomas en lugar de romperse al traducir el texto de la interfaz.
- **Tickets diseñados para flujos de corrección AI:** los issues en Markdown se generan automáticamente con el paquete completo de pruebas adjunto, listos para entregarse directamente a un agente de reparación en lugar de a un triador humano.

Cómo se utiliza en todos los productos (las capacidades que ofrece)

Como **pilar de calidad obligatorio** (el Constitution §11.4.169 designa el submódulo `helix_qa` como uno de los tipos de prueba requeridos), el HelixQA otorga a cada producto de la familia el mismo conjunto de capacidades:

- **Sesiones de QA autónomas:** un simple `helixqa autonomous --project ... --platforms android,desktop,web` libera a un agente LLM con visión que ejecuta aplicaciones reales sin supervisión hasta alcanzar un objetivo de cobertura, generando informes, tickets y vídeos sin intervención humana.
- **Bancos/suites de pruebas:** bancos YAML (piso 219 con ≥ 30), orientados a plataformas, priorizados y trazables línea por línea hasta la documentación que verifican.
- **Pruebas capturadas:** capturas de pantalla, logcat, vídeo, trazas de pila y línea temporal completa —centralizadas y vinculadas desde cada informe, para que cualquier veredicto pueda reproducirse y auditarse a posteriori.
- **Veredictos independientes (§11.4.141 principio de independencia):** su `issuetelector` impulsado por LLM y el oráculo visual juzgan el comportamiento de la aplicación en ejecución de forma independiente al agente que la navegó, eliminando estructuralmente el fallo clásico en el que un sistema valida su propio trabajo como correcto.
- **Compuerta + trinquete de mutación:** `make qa-all / make anti-bluff y challenges/scripts/helixqa_orchestrator_challenge.sh` (8 fases, mutación integrada en §1.1) mantienen demostrada de forma continua la honestidad del HelixQA —y no existe deliberadamente una escotilla de escape `--skip-helixqa` para desactivar la disciplina bajo presión de plazos.

Mayores desafíos técnicos y cómo los resolvimos

- **Evitar falsos positivos en la propia QA** — la herramienta que detecta engaños no debe convertirse en uno → cada paso se valida contra evidencia capturada; un APROBADO sin evidencia se califica como defecto en lugar de como aprobación, y un manifiesto de anclaje de comportamiento vincula cada capacidad anunciada a una prueba ejecutable (CONST-035), de modo que no se puede afirmar ninguna capacidad sin algo que la ponga a prueba.
- **Controlar plataformas heterogéneas desde un único cerebro** — Android, Android TV, Web y Escritorio no comparten modelo de entrada → un único paquete `navigator` abstrae los ejecutores de acciones específicos de cada

plataforma (ADB, Playwright, X11) y los detectores de fallos por plataforma (android/web/escritorio), de manera que la lógica de orquestación se escribe una sola vez y las diferencias entre plataformas quedan en los extremos.

- **Hacer que los agentes autónomos sean útiles, no caóticos** — un LLM no supervisado suelto en una aplicación puede divagar eternamente → LLMsVerifier puntúa y selecciona los modelos adecuados, LLMOrchestrator gestiona los agentes CLI en modo headless (opencode, claude-code, gemini, junie, qwen-code), DocProcessor construye el mapa de características que da un objetivo a la exploración, y VisionEngine mantiene cada decisión anclada en los píxeles reales de la pantalla, no en la imaginación del modelo.
- **Bancos de pruebas seguros para localización** — una suite que codifica texto de interfaz en inglés falla en quince idiomas → los bancos describen solo la estructura, y el texto del prompt visible para el usuario se carga en tiempo de ejecución desde LLM/recursos (CONST-046), de modo que el mismo banco verifica el mismo comportamiento independientemente de la configuración regional.
- **Demostrar que las compuertas no son un engaño** — una compuerta anti-engaño que no puede fallar es el engaño definitivo → mutaciones emparejadas en §1.1 eliminan la captura de evidencia o la aserción anti-engaño de un tipo y exigen que la compuerta FALLE, y un trinquete de mutaciones evita que esa garantía se degrade silenciosamente con el tiempo.

Pila tecnológica

- **Orquestador Go 1.24+** — *por qué*: la QA debe ejecutarse en cualquier lugar donde lo hagan los productos, por lo que un único binario estáticamente enlazado, rápido y portátil supera a una alternativa pesada en tiempo de ejecución; *cómo*: un único binario `cmd/helixqa` CLI que expone subcomandos componibles: `run`, `list`, `report`, `autonomous` y `version`.
- **Bancos de pruebas YAML (pkg/testbank)** — *por qué*: las suites deben ser declarativas y legibles, editables por humanos sin tocar Go; *cómo*: `version/name/test_cases[]` con `id`, `category`, `priority`, `platforms`, `steps[]` ordenados y `documentation_refs[]` para trazabilidad con la documentación de las características.
- **Detectores de fallos/cuelgues (pkg/detector)** — *por qué*: los fallos más críticos son aquellos que ocurren en vivo, durante la interacción, no en una aserción posterior; *cómo*: ADB (`pidof/logcat/screenshot`) para Android y `pgrep` para web/escritorio, monitorizando el proceso mientras la prueba lo controla.
- **Recolección de evidencia (pkg/evidence, pkg/session)** — *por qué*: el contrato anti-engaño solo es real si cada APROBADO está respaldado por pruebas físicas; *cómo*: capturas de

pantalla, logcat, vídeo y trazas de pila capturadas en una línea de tiempo SessionRecorder a la que enlaza cada informe.

- **Sesión autónoma (pkg/autonomous, pkg/navigator, pkg/issuedetector)** — *por qué*: la QA manual exhaustiva en cuatro plataformas no escala, por lo que la exploración debe autogestionarse; *cómo*: un SessionCoordinator de 4 fases junto con ejecutores de acciones (ADB/Playwright/X11) y detección de errores LLM que abarca defectos visuales, de UX, accesibilidad y funcionales.
- **Submódulos externos** — *por qué*: reutilización y desacoplamiento (CONST-051), y —de manera crítica— separación entre el navegador y el evaluador; *cómo*: LLMsVerifier (puntuación de modelos), LLMOrchestrator (agentes CLI en modo headless), VisionEngine (GoCV + LLM Vision), DocProcessor (mapa de características/cobertura), cada uno como componente independiente y con propietario asignado.
- **Compuertas anti-engaño + trinquete de mutaciones** — *por qué*: para exigir a HelixQA el mismo pacto §1.1 que impone a todo lo demás; *cómo*: un escaneo make anti-bluff junto con un manifiesto de anclaje de comportamiento y un trinquete de mutaciones, con helixqa_orchestrator_challenge.sh como validador integral de 8 fases.
- **Matriz de cobertura de 15 filas (docs/test-coverage.md)** — *por qué*: CONST-050(B) exige un conjunto cerrado y completamente contabilizado de tipos de prueba sin lagunas; *cómo*: cada fila está vinculada a un activo ejecutable concreto y a una forma específica de evidencia capturada, de modo que la cobertura es un hecho verificado, no una mera afirmación.

Notas sobre estado y honestidad

- **Estado: beta.** En desarrollo activo (banner de estado del README en la ronda 219). Sometido a su propio estándar anti-engaño.
- **Licencia: Apache-2.0.** Instalación: `go install digital.vasic.helixqa/cmd/helixqa@latest`.

Nivel de prioridad: Helix-principal — un pilar obligatorio de calidad/anti-engaño en la forma en que la familia Helix verifica que las funciones realmente funcionen.